RAG Ultimate Cheat Sheet

Retrieval-Augmented Generation: Concepts, Pipeline & Code

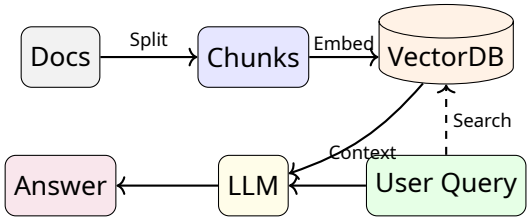
1 Core Concepts

What is RAG?
Technique to optimize LLM output by referencing an authoritative knowledge base outside its training data.

- **Retrieval**: Find relevant info in external data.
- **Generation**: LLM synthesizes answer using that info.

Why RAG?

- **Freshness**: Access real-time/private data.
- **Accuracy**: Reduces hallucinations by grounding.
- **Cost**: Cheaper than fine-tuning.



2 1. Ingestion (Load & Split)

Loaders

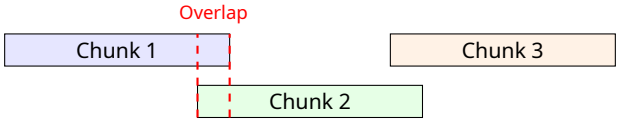
```
from langchain_community.document_loaders \
    import PyPDFLoader, WebBaseLoader

# Load PDF
loader = PyPDFLoader("paper.pdf")
docs = loader.load()

# Load Web
loader = WebBaseLoader("https://...")
docs = loader.load()
```

Splitting (Chunking)
Breaking text into smaller, semantically meaningful pieces.

- **Chunk Size**: Characters per chunk (e.g., 1000).
- **Overlap**: Shared text to maintain context (e.g., 200).




```
from langchain.text_splitter import \
    RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
splits = splitter.split_documents(docs)
```

3 2. Indexing (Embed & Store)

Embeddings
Converts text into dense vectors (lists of numbers). Semantic meaning becomes distance.

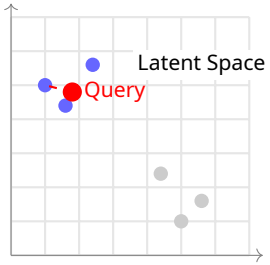
```
from langchain_openai import OpenAIEmbeddings
embeddings = OpenAIEmbeddings()
```

Vector Database
Stores embeddings for fast similarity search.

- **Chroma/FAISS**: Local, open-source.
- **Pinecone/Weaviate**: Managed cloud.

```
from langchain_chroma import Chroma

# Create DB from splits
vectorstore = Chroma.from_documents(
    documents=splits,
    embedding=embeddings
)
```



4 3. Retrieval Strategies

Basic Retriever

```
retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 4} # Top 4 docs
)
docs = retriever.invoke("What is RAG?")
```

Search Types

- **Similarity**: Cosine/Euclidean distance.
- **MMR**: Maximal Marginal Relevance. Balances relevance with diversity to avoid redundant chunks.

Multi-Query (Query Expansion)
Generates multiple versions of user query to catch different phrasings.

```
from langchain.retrievers.multi_query \
    import MultiQueryRetriever

retriever = MultiQueryRetriever.from_llm(
    retriever=vectorstore.as_retriever(),
    llm=llm
)
```

5 4. Generation (The Chain)

Prompt Template

```
from langchain_core.prompts import ChatPromptTemplate

template = """Answer based only on context:
{context}

Question: {question}
"""
prompt = ChatPromptTemplate.from_template(template)
```

LCEL Chain

```
from langchain_core.output_parsers \
    import StrOutputParser
from langchain_core.runnables \
    import RunnablePassthrough

def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

chain = (
    {"context": retriever | format_docs,
     "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
```

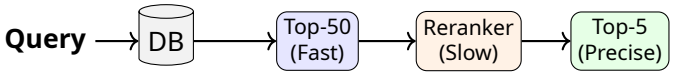
```
)

chain.invoke("How does RAG work?")
```

6 Advanced RAG Techniques

Reranking

Two-stage process: 1. Fetch many docs (e.g., 50) quickly. 2. Use a high-precision Cross-Encoder to re-order them.




```
from langchain.retrievers import \
    ContextualCompressionRetriever
from langchain.retrievers.document_compressors \
    import CrossEncoderReranker

compressor = CrossEncoderReranker(
    model=HuggingFaceCrossEncoder(
        model_name="BAAI/bge-reranker-base"
    ), top_n=5
)
```

```
rerank_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=retriever
)
```

Parent Document Retriever

Fetch small chunks for better embedding match, but return the larger parent chunk to LLM for context.

Hybrid Search

Combines **Keyword Search** (BM25) + **Vector Search**. Good for exact matches (acronyms, IDs).

7 Evaluation (RAGAS)

Metrics

- **Faithfulness**: Is answer derived from context? (No hallucinations).
- **Answer Relevance**: Does answer address the query?
- **Context Precision**: Are relevant chunks ranked high?
- **Context Recall**: Is all necessary info retrieved?

```
from ragas import evaluate
from ragas.metrics import \
    faithfulness, answer_relevancy

results = evaluate(
    dataset,
    metrics=[faithfulness, answer_relevancy]
)
```

8 Common Pitfalls

- **Lost in Middle**: LLMs focus on start/end of context. Reorder highly relevant chunks to edges.
- **Bad Splitting**: Breaking sentences mid-thought. Use semantic chunking.
- **Retrieval Fail**: Relevant info exists but vector distance is large. Use Hybrid search.